

Watch-Outs & Gaps vs. Industry Standards

Design Watch-Outs

Generic CRUD DatabaseAdapter

- **rawQuery escape hatch** — optional method for performance-critical multi-table operations (IAM permission chain JOINS). Adapters that don't implement it fall back to multiple `findMany` calls. Design decision: the generic CRUD is for simple ops and plugins; core IAM queries can use `rawQuery` for performance.
- **create return type `Promise<T>` is unconstrained** — no type relationship between `model: 'user'` and return type `T`. Intentional trade-off (same as Better Auth). The internal adapter layer handles typing. Document as a known limitation.

Plugin System

- **Plugin ordering** — hooks run in registration order. Document this clearly. Consider if any hooks need explicit priority.

Plugin Runtime Integration

- **wrapAdapter and scopeRules are not wired into request handling.** The building blocks exist and are tested (`chainAdapterWrappers`, `collectScopeRules`, `wrapAdapterWithScopeRules` in `src/core/plugin-runner.ts`), but the Hono middleware (`src/hono/middleware/auth.ts`) does not call them per-request. Plugins declaring `wrapAdapter` (e.g., tenancy) or `scopeRules` (e.g., data-isolation) will not take effect at runtime until the middleware applies them. **Fix:** After JWT verification in the auth middleware, call `chainAdapterWrappers` with registered plugins, then `collectScopeRules` for the authenticated user, and expose the scoped adapter on the Hono context.

Permission Evaluation

- **Performance at scale** — evaluation with conditions requires DB queries per check. Plan caching (per-request cache, short TTL cache of user permissions). Use `rawQuery` for the permission chain JOIN when available.
- **Wildcard permissions** — no `*` wildcards (e.g., `post:*` for all actions). AWS and GCP support these. Decide upfront.
- **Hierarchical resources** — GCP has `projects/*/buckets/*/objects/*`. Not needed now, but keep the door open.

Data Isolation Plugin

- **Bulk operations** — `update` and `delete` with scope rules must be tested carefully to prevent cross-tenant mutations.

Password Hasher

- WASM Argon2id is slower than native. Document the trade-off and make `@node-rs/argon2` / `Bun.password` swaps dead simple for users who don't need edge runtime.

General Risks

- **Adapter testing** — the generic CRUD contract needs thorough integration tests against real databases. Subtle adapter bugs is what killed Lucia. Mitigated by vitest + bun:sqlite in-memory for unit tests + testcontainers PostgreSQL for integration tests.
- **Scope creep** — the plugin list is ambitious (8 plugins). **Ship core + Drizzle adapter first, validate the plugin interface, then build plugins incrementally.**
- **Documentation** — Better Auth's biggest weakness is docs lagging features. Plan docs alongside code.

- **Soft deletes** — loyalbook uses `rowStatus`, tdmp uses `isActive`. Fortress core uses hard deletes. Soft deletes are a consumer concern (use `update` instead of `delete`). Document this decision.
- **createdBy / audit trail** — both source projects have `createdBy` on most tables. The generic CRUD adapter doesn't know about the authenticated user. Service layer explicitly passes `createdBy` in the `data` object. Not magic, not hidden — explicit and clear.

Gaps vs. Industry Standards

Gap	Who Has It	Priority	Notes
Password policy & breach checking	Auth0, Clerk, Keycloak, Better Auth	Critical	No password validation anywhere. Configurable min/max length (NIST 800-63B: 8/128). HIBP k-anonymity API to reject breached passwords. Hook into <code>beforeRegister</code> and password reset. Config: <code>passwordPolicy?: { minLength?, maxLength?, checkBreached? }.</code> New file: <code>src/core/auth/password-policy.t</code>
Rate limiting	Keycloak, Auth0, Clerk	Critical	Critical for login/token endpoints. Build as plugin. <code>RateLimitError</code> (429) already in error hierarchy.
Account lockout	Most managed services	High	Lock after N failed login attempts. Simple plugin using <code>hooks.beforeLogin</code> . Exponential backoff (15min → 30min → 1hr). Required for SOC 2, ISO 27001, HIPAA.

Gap	Who Has It	Priority	Notes
Session/device management	Auth0, Clerk, Keycloak, Better Auth	High	Refresh tokens store IP/userAgent but no list/revoke API. Add <code>listSessions(userId)</code> , <code>revokeSession(tokenId)</code> , <code>revokeAllOtherSessions(userId, currentTokenId)</code> . Enrich refresh token with <code>deviceName?</code> , <code>lastActiveAt</code> . Modify <code>auth-service.ts</code> .
Token fingerprinting on refresh	Auth0, Clerk	High	Fortress stores IP/userAgent but doesn't validate on refresh. Optionally compare fingerprint on token refresh. Config: <code>jwt.validateRefreshFingerprint? boolean \ 'warn'</code> . 'warn' mode logs mismatch but allows (mobile users); <code>true</code> rejects. Prevents stolen token reuse.
CSRF explicit strategy	Auth.js, Better Auth, Keycloak	High	Architecture mentions <code>SameSite</code> cookies via <code>responseHeaders</code> but no explicit CSRF token pattern. Provide <code>csrfMiddleware()</code> for Hono. Document that <code>SameSite=Strict</code> + custom header requirement is sufficient for modern browsers. New files: <code>src/hono/middleware/csrf.ts</code> , <code>docs/security.md</code> .
Audit logging	Keycloak, Ory	High	Auth events (login, failed login, permission denied, role changes). Plugin using hooks. Immutable event log, 2-year retention recommended. Required for SOC 2, HIPAA, PCI-DSS.

Gap	Who Has It	Priority	Notes
Admin impersonation	Keycloak, Auth0, WorkOS, Ory	Medium	Admin acts as user without knowing password. Method: <code>fortress.auth.impersonate(adminId, targetUserId)</code> . Returns scoped token with <code>impersonatedBy</code> claim + shorter expiry (30min). Requires <code>fortress:impersonate</code> permission. Audit log entry.
Webhooks plugin	Auth0, Clerk, WorkOS, Ory	Medium	Not in architecture.md. Notify external systems on auth events. <code>fortress_webhook</code> model. HMAC-SHA256 signed payloads, delivery retries (3x exponential backoff). New plugin: <code>src/plugins/webhook/index.ts</code> .
isSystem flag on roles	Keycloak	Medium	Prevents accidental deletion of seeded roles. Add <code>isSystem: boolean</code> (default false) to role model. <code>deleteRole</code> throws if <code>isSystem === true</code> . Roles from <code>sync:push</code> marked as system. Modify <code>schema.ts</code> , <code>iam-service.ts</code> , <code>resource-sync.ts</code> .
WebAuthn / Passkeys	Auth.js, Clerk, @oslojs/webauthn	Medium	Growing fast. Plugin using routes + models. Architecture validated — no gaps.
Magic link auth	Better Auth, Auth.js	Low	Easy plugin — same pattern as email verification but issues tokens.
Session management (stateful)	Auth.js, Lucia (was), Better Auth	Low	JWT + refresh tokens is valid. Cookie handling via <code>AfterHookContext.responseHeaders</code> .

Gap	Who Has It	Priority	Notes
SCIM (directory sync)	WorkOS, Okta	Low	Enterprise feature for syncing users from external directories. Not needed initially.

Library Authoring Issues

Reviewed 2026-04-03 against TypeScript library authoring best practices.

P1 — High

No npm Publishing Path

- JSR-only. No build step, no `dist/` output, no compiled `.js` + `.d.ts` files.
- `package.json` has no `exports` field, no `main`, no `types`. The `module: "src/index.ts"` field is a Bun convention, not a Node standard.
- npm is where 95%+ of the TS ecosystem lives. JSR adoption is still small.
- **Fix:** Add `tsup` for ESM + CJS bundles with declarations. Add `exports` map to `package.json`. Add `prepublishOnly` script. CI workflow: publish to both JSR and npm on git tag. New files: `tsup.config.ts`, `.github/workflows/publish.yml`.

No Security Documentation

- No `docs/security.md` exists. Recommended CSRF strategy, JWT secret requirements, rotation procedure, password hashing guide, rate limiting deployment patterns, token storage best practices (`httpOnly` cookies vs `localStorage`), HTTPS requirements — none documented.
- **Fix:** Create `docs/security.md` covering all security recommendations.

CLI Tool Not Implemented

- `architecture.md` references `sync:push`, `sync:pull`, `sync:types` commands but no CLI exists.
- **Fix:** Create `bin/fortress.ts` with commands: `init` (scaffold config, `.env` template, `fortress.resources.json`), `sync:push`, `sync:pull`, `sync:types`, `generate-secret` (64-byte cryptographically random hex).

P2 — Medium

update Return Type on No-Match Is Undefined Behavior

- The adapter contract says “may return undefined or the unchanged input” when no rows match.
- A contract with undefined behavior at its boundaries is not a contract.
- **Fix:** Change return type to `Promise<T | null>` (null = no match). Update adapter conformance tests to verify null on no-match.

InferPlugins Utility Type Is Never Used

- `src/core/plugin.ts` defines `InferPlugins` but the `Fortress` interface doesn’t use it. `fortress.plugins.myPlugin.my` has no type safety.
- **Fix:** Wire `InferPlugins` into the `Fortress` type so plugin methods are typed.

No README

- JSR and npm both surface README as primary documentation. Without one, the library has no public-facing docs.
- `CLAUDE.md` is an AI context file, not user documentation.

No CHANGELOG, Release Process, or Security Policy

- Version 0.0.1 with no CHANGELOG, no release workflow in CI, no conventional commits.
- No `SECURITY.md` for vulnerability disclosure.
- For an auth library where security patches must be communicated clearly, this is a significant gap.
- **Fix:** Create `README.md` (quick start, API overview, plugin list), `CHANGELOG.md` (start at v0.1.0), `SECURITY.md` (disclosure process). Enforce conventional commits via commitlint. Semantic versioning.

P3 — Low

`moduleResolution: "bundler"` Is Wrong for a Library

- `tsconfig.json` uses `"bundler"` resolution, which allows extensionless imports that fail with Node's native ESM.
- **Fix:** Use `"node16"` or `"nodenext"` for library code.

No Drizzle Adapter Isolation Tests

- No tests for PostgreSQL dialect path, `buildWhereCondition` error cases, or `sanitizeForSqlite`.
- Tested only indirectly through SQLite integration tests.

Unconstrained DatabaseAdapter Generics

- `create<T>`, `findOne<T>` etc. have no link between the `model` string and return type `T`. You can write `db.findOne<FortressUser>({ model: 'refresh_token' })` with no compiler error.
- **Fix:** Consider a mapped type linking model names to their shapes, or document as an intentional trade-off.